

Vula OS Group

Internet-Draft

Intended status: Experimental (proposal)

Vula OS Group

July 2026

Work Request & Assignment Protocol (WRAP)

draft-wrap-00

ABSTRACT

WRAP is an open protocol for coordinating work between independent parties without a central operator. One party issues a *work order*; it is offered to one or more *pools*; performers respond; the issuer assigns exactly one performer; the performer reports progress; and completion is recorded as a signed *attestation* that neither side can later rewrite. Identity is an Ed25519 keypair, so a worker's history and reputation belong to the worker rather than to any platform. All state is a set of signed, content-addressed objects merged by a deterministic CRDT algebra, so participants converge without coordination and keep working while offline or partitioned. WRAP deliberately avoids the hard problems it does not need: assignment is decided by the issuer, who is already the natural authority, so there is no consensus protocol, no leader election, and no distributed lock. The core is domain-neutral — courier delivery, skilled trades, field service, and volunteer dispatch are *profiles* over one object model. WRAP composes existing standards (Ed25519, CBOR, COSE, HLC, RFC 6962 hash chains) and defines two transport bindings: plain HTTP, and the DMTAP substrate. Neither is required by the other; the core depends on neither.

STATUS OF THIS MEMO

This Internet-Draft is submitted in full conformance with the provisions of an open, royalty-free specification. Internet-Drafts are working documents; this document is a proposal and is *not* an RFC — an RFC number is assigned only upon publication by the relevant standards body. This is **version 0 and carries no compatibility guarantee**: it describes an implementation in progress and may be updated, replaced, or obsoleted at any time. It is inappropriate to use this document as reference material or to cite it other than as “work in progress.”

REQUIREMENTS LANGUAGE

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY**, and **OPTIONAL** in this document are to be interpreted as described in BCP 14 (RFC 2119, RFC 8174) when, and only when, they appear in all capitals, as shown here.

Table of Contents

1. Overview

- 1.1. The one idea
- 1.2. What WRAP is
- 1.3. What WRAP is not
- 1.4. Design constraints
- 1.5. Document status

2. Identity

- 2.1. The key is the identity
- 2.2. Roles
- 2.3. Key names
- 2.4. What WRAP does not define

3. Object model

- 3.1. Overview
- 3.2. Common header
- 3.3. WorkOrder (`0x01`)
- 3.4. Offer (`0x02`)
- 3.5. Bid (`0x03`)
- 3.6. Assignment (`0x04`)
- 3.7. Progress (`0x05`)
- 3.8. Attestation (`0x06`)
- 3.9. Place
- 3.10. Window
- 3.11. Compensation

4. Wire format

- 4.1. Encoding
- 4.2. Versioning
- 4.3. Object identifier
- 4.4. Unknown fields and unknown kinds
- 4.5. Reserved and forbidden keys
- 4.6. Sizes

5. Signing

- 5.1. Sign the object, not the frame
- 5.2. Preimage
- 5.3. Signature envelope
- 5.4. Verification
- 5.5. Authorship rules
- 5.6. Referential validation

6. Lifecycle

- 6.1. States
- 6.2. Transitions
- 6.3. Computing current state
- 6.4. Timeouts

7. Merge

- 7.1. The algebra
- 7.2. Hybrid logical clock
- 7.3. Tie-break
- 7.4. Convergence
- 7.5. Compaction
- 7.6. Partition behaviour

8. Pools and discovery

- 8.1. There is no discovery without infrastructure
- 8.2. Three paths
- 8.3. What a pool does and does not do
- 8.4. Membership
- 8.5. Privacy
- 8.6. Why not gossip

9. Trust

- 9.1. WRAP is not trustless
- 9.2. Curated pools
- 9.3. Weighting attestations
- 9.4. Reputation is inverted
- 9.5. Append-only publication
- 9.6. Disputes

10. Fulfilment

- 10.1. The problem
- 10.2. Handoff codes
- 10.3. Proof map
- 10.4. Weaker proofs
- 10.5. Partial and multi-visit work

11. Transport bindings

- 11.1. Binding A — HTTP (REQUIRED)
- 11.2. Binding B — DMTAP substrate (OPTIONAL)
- 11.3. Other bindings

12. Profiles

- 12.1. What a profile is
- 12.2. `delivery/v0`

12.3. `trades/v0`

12.4. Other applicable domains

12.5. Registering a profile

13. Errors

13.1. Codes

13.2. Reporting

13.3. Silence

14. Security considerations

14.1. What WRAP protects

14.2. What WRAP does not protect

14.3. Misbehaving parties

14.4. Replay

14.5. Denial of service

14.6. Privacy of beneficiaries

15. Conformance

15.1. Vectors are normative

15.2. Required coverage

15.3. The cases most likely to be got wrong

15.4. Interoperability

16. References

16.1. Normative

16.2. Informative

16.3. Acknowledgements

1. Overview

1.1. The one idea

Every platform that coordinates work — delivery apps, gig marketplaces, trade directories, field-service dispatchers — solves the same problem the same way: put one company in the middle, let it hold every identity, every job record and every reputation, and let it take a cut for the privilege. The coordination is not the hard part. The *centralization* is not a technical necessity; it is a business model that arrived wearing the costume of an architecture.

Look at what actually forces a central operator, and it comes down to one thing: **somebody must decide who gets the job**. Nearly every platform runs first-come-claim — the job is broadcast, workers race, and the first tap wins. A race needs an arbiter. An arbiter is a central server. Everything else — identity, reputation, payments, history — gets absorbed into that server because it is already there.

WRAP removes the race. The party who wants the work done decides who does it:

The issuer assigns. The only contended decision in the protocol is resolved by the party who is already its natural authority — the one who issued the work. Whoever cooked the food decides who carries it.

That single choice eliminates consensus, leader election, and distributed locking from the protocol. What remains is a set of signed objects that merge deterministically, which any two participants can exchange over any transport, in any order, with arbitrary delay, and still converge.

1.2. What WRAP is

WRAP defines:

- an **identity** model — an Ed25519 keypair, held by the participant, not by a platform (§2);
- an **object model** — `WorkOrder`, `Offer`, `Bid`, `Assignment`, `Progress`, `Attestation` (§3);
- a **canonical encoding** and signing rule, so a signed object is valid on any transport (§4, §5);
- a **lifecycle** state machine (§6);
- **merge semantics** — a small CRDT algebra with a total order, so replicas converge without coordination (§7);
- **pools** — how work reaches performers without a privileged node (§8);
- a **trust** model that is honest about Sybil resistance (§9);
- **fulfilment** — how completion is proven to a party who was not present (§10);
- two **transport bindings**, neither required (§11);
- **profiles** — delivery, trades, and how to define others (§12).

1.3. What WRAP is not

- **Not a payment system.** A work order carries compensation *terms* as data. Settlement happens out of band, by whatever means the parties already use. There is no escrow, no currency, and no blockchain anywhere in this document.
- **Not a mesh.** Broadcasting work orders to every participant would publish customer addresses and commercial volumes to strangers, invite spam, and scale badly for something whose scope is usu-

ally a few kilometres. WRAP is point-to-point plus pools (§8).

- **Not trustless.** Open participation without a gatekeeper admits Sybils. WRAP does not claim to have solved this; it makes the trust source explicit and replaceable (§9).
- **Not a governance framework.** WRAP says nothing about how a pool decides its rules, prices, or membership. That is deliberate and it is the most important thing WRAP leaves to its users.
- **Not a scheduler or optimiser.** Route planning, batching, and matching heuristics are implementation concerns, not wire concerns.
- **Not anonymous.** Participants are identified by public key to their counterparties. WRAP protects *integrity* and *ownership of history*, not metadata privacy.

1.4. Design constraints

These constraints produced the design and explain most of its choices.

It must work offline. A courier goes through a tunnel; a plumber works in a basement; a café's connection fails during the dinner rush. Participants **MUST** be able to make progress while partitioned and converge afterwards without losing work. This forces CRDT merge and forbids any synchronous coordination.

It must run on modest hardware. The reference deployment is a low-cost Android tablet and a phone. This forbids heavyweight consensus and rules out requiring a always-on server per participant.

It must be implementable in an afternoon. A protocol that requires adopting a novel messaging stack acquires zero independent implementations. WRAP requires Ed25519, CBOR and an ordinary HTTP client. Nothing else.

The worker's history must be portable. If reputation lives in a platform's database, the platform owns the worker. WRAP makes attestations signed objects held by the parties, so a worker who leaves a pool takes their record with them.

No privileged nodes. Some infrastructure is unavoidable for open discovery (§8.1 is explicit about why). WRAP requires only that any such node be *replaceable*: anyone may run one, participants may use several, and losing one costs reachability, never identity, history, or data.

1.5. Document status

This is **version 0**. It carries no compatibility guarantee and describes an implementation in progress. It is written with the rigour of a standards document because that discipline produces better designs, not because stability is being claimed.

The conformance vectors (§15) are normative where they disagree with the prose. This is deliberate: prose cannot tell an implementer that they got a tie-break wrong, and the parts of comparable protocols that actually achieved independent implementations are precisely those that shipped vectors.

2. Identity

2.1. The key is the identity

A WRAP participant is an **Ed25519 keypair**. The public key is the identity. There is no account, no registration, and no issuing authority. A participant **MAY** generate a keypair offline and begin issuing

or accepting work immediately.

A **Principal** is a 32-byte Ed25519 public key. Wherever this document says “issuer”, “performer”, “pool”, or “attestor”, it means a Principal.

WRAP defines no username, no email address, and no directory. Display names are a presentation concern and **MUST NOT** be used for routing, authorization, or identity comparison. Two Principals are equal if and only if their public keys are byte-equal.

Note. This makes WRAP identities compatible by construction with any system whose root identity is also an Ed25519 key — including DMTAP, whose identity key `IK` is Ed25519. A DMTAP participant is already a valid WRAP Principal. WRAP does not depend on DMTAP for this to be true (§11.2).

2.2. Roles

Roles are not properties of a Principal; they are positions in a relationship. The same key **MAY** be an issuer in one work order and a performer in another — a plumber who subcontracts is both.

Role	Meaning
Issuer	Creates a <code>WorkOrder</code> and decides its <code>Assignment</code> .
Performer	Executes assigned work and reports <code>Progress</code> .
Pool	Distributes <code>Offer</code> s to a membership. Holds no authority over the work itself.
Attestor	Signs an <code>Attestation</code> about an outcome. Usually the issuer, the performer, or a beneficiary.

A Pool is a Principal like any other. It has no protocol powers: it cannot assign, cannot complete, and cannot alter a work order. It is a distribution convenience (§8) and nothing more. This is the property that keeps pools replaceable.

2.3. Key names

Raw public keys are unusable by humans. Implementations **SHOULD** render a Principal as an **8-word key name** derived from its public key:

```
words(BLAKE3-256(pubkey)) over a 1024-word list, 8 words
```

Each word contributes 10 bits, for 80 bits of the digest — enough that two participants reading words aloud can confirm they hold the same key.

Key names are a **display and verification** convention, not a namespace. They are not looked up, not registered, and not resolved. An implementation **MUST NOT** accept a key name as a substitute for a public key on the wire.

Implementations **MAY** additionally show a local **petname** — a nickname the user assigns to a key on their own device. Petnames are local, never transmitted, and never authoritative.

2.4. What WRAP does not define

Key rotation and revocation are out of scope for v0. This is a known gap and it is stated plainly rather than hidden: a participant who loses their key loses their identity and their accumulated attestations, and there is currently no in-protocol recovery.

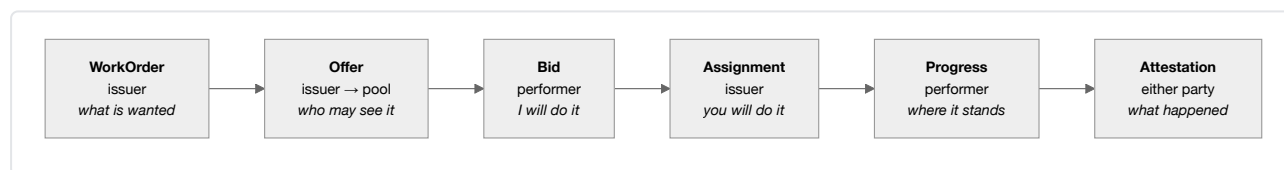
Deployments that need rotation **SHOULD** layer an existing mechanism — a device certificate chain, a key transparency log, or an out-of-band re-enrollment through their pool — beneath WRAP. A future version **MAY** define one. It is not defined here because a rotation scheme without a matching revocation-checking rule fails open, and a fail-open identity mechanism is worse than an absent one.

Until then, the practical mitigation is that attestations (§10) are held by *both* parties and by the pool. A worker who must re-key can present their old attestations, signed by counterparties who still exist, and ask a pool to vouch for the continuity. That is a social recovery path, not a cryptographic one, and implementations **SHOULD NOT** describe it as more than that.

3. Object model

3.1. Overview

WRAP defines six object kinds. Every object is signed by its author, content-addressed, and immutable once created. State changes are expressed by *adding objects*, never by mutating them — which is what makes merge trivial and history non-repudiable.



Kind	Value	Author	Merge (§7)
WorkOrder	0x01	Issuer	Immutable; amendments are new objects
Offer	0x02	Issuer	Add-only set
Bid	0x03	Performer	Add-only set, withdrawable
Assignment	0x04	Issuer only	Last-writer-wins, single writer
Progress	0x05	Assigned performer	Add-only log
Attestation	0x06	Any party	Add-only set

Kinds 0x40 – 0x7f are reserved for profile-specific objects (§12). Kinds 0x80 and above are reserved for future core use. An implementation encountering an unknown kind **MUST** ignore it silently (§4.4).

3.2. Common header

Every object carries the same first five fields.

Key	Name	Type	Notes
0	—	—	FORBIDDEN. See §4.5.

Key	Name	Type	Notes
1	<code>v</code>	uint	Format version. <code>0</code> for this document.
2	<code>kind</code>	uint	Object kind from §3.1.
3	<code>id</code>	bstr	<code>0x1e BLAKE3-256(canonical body)</code> (§4.3).
4	<code>author</code>	bstr(32)	Ed25519 public key of the signer.
5	<code>ts</code>	tstr	Hybrid logical clock stamp (§7.2).

`id` is computed over the canonical encoding of the object *excluding* keys 3 and the signature, so it is stable and self-verifying.

3.3. WorkOrder (`0x01`)

The unit of work. Immutable once issued.

Key	Name	Type	Req	Notes
6	<code>profile</code>	tstr	MUST	e.g. <code>"delivery/v0"</code> , <code>"trades/v0"</code> (§12)
7	<code>title</code>	tstr	MUST	Short human summary
8	<code>detail</code>	tstr	MAY	Free text
9	<code>places</code>	array	MAY	Ordered <code>Place</code> list (§3.9)
10	<code>window</code>	map	MAY	<code>Window</code> (§3.10) — when it may happen
11	<code>comp</code>	map	MAY	<code>Compensation</code> (§3.11) — terms only, never settlement
12	<code>needs</code>	array of tstr	MAY	Capability requirements, e.g. <code>"vehicle:bicycle"</code>
13	<code>expires</code>	uint	MUST	Unix seconds. After this, no new <code>Bid</code> or <code>Assignment</code> is valid.
14	<code>refs</code>	map	MAY	Opaque external identifiers, e.g. <code>{"order":"BB-4417"}</code>
15	<code>body</code>	map	MAY	Profile-specific fields (§12.1)

A `WorkOrder` MUST NOT be edited. To change terms, the issuer issues a new `WorkOrder` referencing the old one in `refs` and cancels the original with a `Progress` object (§3.7).

`expires` is REQUIRED and has no default. A pool full of work orders that never expire is indistinguishable from a pool of stale work orders, and performers stop trusting all of it.

3.4. Offer (`0x02`)

Binds a work order to an audience. One `Offer` per pool; several MAY exist for one work order.

Key	Name	Type	Req	Notes
6	<code>order</code>	bstr	MUST	<code>WorkOrder.id</code>
7	<code>pool</code>	bstr(32)	MUST	Pool Principal, or the issuer's own key for a direct offer
8	<code>mode</code>	uint	MUST	<code>0</code> direct, <code>1</code> open bid, <code>2</code> sealed bid
9	<code>closes</code>	uint	MAY	Unix seconds; bidding window. Defaults to <code>WorkOrder.expires</code> .

A direct offer (`mode = 0`) names its intended performer in `needs` on the work order or by convention in the pool; it exists so that the common case — a shop and its own driver, a homeowner and

their regular plumber — requires no pool infrastructure at all.

3.5. Bid (0x03)

A performer's response. Optional in direct mode.

Key	Name	Type	Req	Notes
6	<code>order</code>	bstr	MUST	<code>WorkOrder.id</code>
7	<code>offer</code>	bstr	MUST	<code>Offer.id</code> the bid answers
8	<code>quote</code>	map	MAY	<code>Compensation</code> (§3.11) if the performer names terms
9	<code>eta</code>	uint	MAY	Unix seconds, performer's estimate
10	<code>note</code>	tstr	MAY	Free text
11	<code>withdrawn</code>	bool	MAY	<code>true</code> retracts an earlier bid by the same author

Bids are an **add-only set**. Concurrent bids from performers who cannot see each other all survive the merge; a bid is never lost because two people bid at the same moment. Withdrawal is a new `Bid` object with `withdrawn = true` and the same `order` and `author`; the pair is resolved at read time by `ts`.

3.6. Assignment (0x04)

The issuer's decision. **The issuer is the only valid author.** An `Assignment` whose `author` is not the `WorkOrder.author` MUST be rejected with `ERR_NOT_ISSUER` (§13).

Key	Name	Type	Req	Notes
6	<code>order</code>	bstr	MUST	<code>WorkOrder.id</code>
7	<code>performer</code>	bstr(32)	MUST	Assigned Principal
8	<code>terms</code>	map	MAY	Agreed <code>Compensation</code> ; defaults to the accepted bid's quote
9	<code>revoked</code>	bool	MAY	<code>true</code> unassigns (performer no-show, cancellation)

Because there is exactly one legitimate writer, `Assignment` merges by last-writer-wins on `ts` with no ambiguity. This is the entire reason WRAP needs no consensus: the contended decision has a single authorized author, so “concurrent conflicting assignments” is not a state the protocol can reach between honest participants.

A malicious issuer *can* sign two assignments. That is not a distributed systems failure — it is a party misbehaving in a way that is signed, permanent, and attributable. §14.3 discusses the consequences.

3.7. Progress (0x05)

An append-only event on an assigned work order.

Key	Name	Type	Req	Notes
6	<code>order</code>	bstr	MUST	<code>WorkOrder.id</code>
7	<code>state</code>	tstr	MUST	Lifecycle state (§6)
8	<code>at</code>	map	MAY	<code>Place</code> (§3.9), if the event is located

Key	Name	Type	Req	Notes
9	<code>note</code>	tstr	MAY	Free text
10	<code>body</code>	map	MAY	Profile-specific

Valid authors are the assigned performer and the issuer. Progress objects are never removed; a correction is a later `Progress`, and read-time state is the highest-`ts` event that is reachable in the state machine (§6.3).

3.8. Attestation (`0x06`)

A signed claim about an outcome. This is what makes reputation portable.

Key	Name	Type	Req	Notes
6	<code>order</code>	bstr	MUST	<code>WorkOrder.id</code>
7	<code>subject</code>	bstr(32)	MUST	Principal the claim is about
8	<code>outcome</code>	uint	MUST	<code>0</code> completed, <code>1</code> failed, <code>2</code> cancelled, <code>3</code> disputed
9	<code>rating</code>	uint	MAY	<code>1</code> – <code>5</code>
10	<code>proof</code>	map	MAY	Fulfilment proof (§10)
11	<code>note</code>	tstr	MAY	Free text

An attestation is meaningful only in the light of who signed it (§9.3). Implementations MUST NOT aggregate attestations without regard to author; a self-signed five-star rating is worth exactly nothing and MUST NOT be counted.

3.9. Place

Key	Name	Type	Notes
1	<code>role</code>	tstr	e.g. <code>"pickup"</code> , <code>"dropoff"</code> , <code>"site"</code>
2	<code>lat</code>	float	WGS84
3	<code>lon</code>	float	WGS84
4	<code>label</code>	tstr	Human address
5	<code>detail</code>	tstr	Access notes, unit number, gate code
6	<code>geo</code>	tstr	Optional GeoJSON for areas rather than points

3.10. Window

Key	Name	Type	Notes
1	<code>earliest</code>	uint	Unix seconds
2	<code>latest</code>	uint	Unix seconds
3	<code>duration</code>	uint	Expected seconds of work
4	<code>kind</code>	uint	<code>0</code> immediate, <code>1</code> scheduled appointment

`kind = 1` is what makes non-delivery work expressible. A plumber booked for Thursday between 09:00 and 12:00 is an ordinary work order, not a special case.

3.11. Compensation

Terms only. WRAP never moves money (§1.3).

Key	Name	Type	Notes
1	<code>currency</code>	tstr	ISO 4217, e.g. <code>"ZAR"</code>
2	<code>amount</code>	int	Minor units (cents). Fixed component.
3	<code>rate</code>	int	Minor units per <code>unit</code>
4	<code>unit</code>	tstr	e.g. <code>"km"</code> , <code>"hour"</code>
5	<code>max</code>	int	Cap on the variable component
6	<code>note</code>	tstr	e.g. <code>"materials billed separately"</code>

4. Wire format

4.1. Encoding

Objects are encoded as **CBOR** (RFC 8949) maps with **unsigned integer keys**.

Integer keys are chosen over text keys for three reasons: they are compact on constrained links, they are stable under translation and refactoring, and they force a registry discipline — a new field requires allocating a number, which is a visible act, rather than inventing a string, which is not.

Implementations **MUST** use **deterministic encoding** as defined in RFC 8949 §4.2.1:

- map keys sorted in ascending numeric order;
- shortest-form integer encoding;
- definite-length arrays and maps only;
- no duplicate keys;
- no indefinite-length strings.

Two implementations encoding the same object **MUST** produce byte-identical output. This is not a stylistic preference: the object identifier and the signature are both computed over these bytes, so any encoding freedom is a correctness bug.

4.2. Versioning

Key 1 (`v`) is the format version and **MUST** be present. This document defines version `0`.

An implementation receiving an object with a `v` it does not support **MUST** ignore the object. It **MUST NOT** attempt best-effort interpretation: a work order whose semantics have changed is worse than a work order that was never seen.

4.3. Object identifier

```
id = 0x1e || BLAKE3-256( canonical_bytes )
```

where `canonical_bytes` is the deterministic CBOR encoding of the object map with key 3 (`id`) and the signature omitted, and `0x1e` is the multihash prefix for BLAKE3-256.

The identifier is therefore self-verifying: a receiver recomputes it and rejects any mismatch with `ERR_BAD_ID` (§13). Content addressing gives deduplication for free — the same object relayed by three peers is stored once.

4.4. Unknown fields and unknown kinds

Two rules, both load-bearing for forward compatibility:

Unknown keys MUST be preserved and ignored. An implementation that receives an object containing keys it does not understand MUST NOT drop them when re-encoding or relaying, because doing so would invalidate the signature and silently corrupt the object for everyone downstream. It MUST NOT reject the object either.

Unknown kinds MUST be ignored silently. Not acknowledged, not rejected, not errored. This lets a deployment introduce a new object kind without breaking older peers, which is what makes profiles (§12) additive rather than forking.

4.5. Reserved and forbidden keys

Key 0 is FORBIDDEN in every object. An object containing key 0 MUST be rejected with `ERR_FORBIDDEN_KEY` (§13).

This is a deliberate trap rather than a gap. Key 0 is the value an encoder produces when a field name fails to resolve to a registered number — the most likely silent-corruption bug in an integer-keyed format. Reserving it as an error makes that bug loud on the first message instead of subtly wrong forever.

Keys 16–31 are reserved for future core use in all object kinds. Keys 32 and above are available to profiles (§12.1) and MUST NOT be assigned by this document.

4.6. Sizes

An implementation MUST reject any object larger than **65 536 bytes** with `ERR_T00_LARGE`. Work orders describe work; they do not carry payloads. Photos, signatures and documents referenced by an attestation MUST be carried by reference (a URL or content address), never inline.

5. Signing

5.1. Sign the object, not the frame

Every WRAP object is signed by its author over a canonical preimage derived from the object itself, **never over a transport frame**.

This is the single decision that makes transport bindings cheap. Because the signature covers the object bytes, the same signed object is valid whether it arrived as an HTTP request body, a row in a sync batch, a line in a file on a USB stick, or an operation in a CRDT log. A binding becomes a framing detail rather than a re-signing exercise, and an object can be relayed by an untrusted intermediary without losing its authenticity.

5.2. Preimage

```
preimage = "WRAP-v0/object" || 0x00 || canonical_bytes
```

where `canonical_bytes` is exactly the bytes hashed for `id` (§4.3) — the deterministic CBOR encoding of the object with key 3 and the signature omitted.

The domain-separation tag and the `0x00` terminator are REQUIRED. Without domain separation, a signature produced for a WRAP object could be replayed as a valid signature in another protocol that happens to sign raw CBOR, and vice versa.

5.3. Signature envelope

A signed object is transmitted as a two-element CBOR array:

```
[ canonical_bytes : bstr, signature : bstr(64) ]
```

`signature` is Ed25519 (RFC 8032) over `preimage`, produced by the private key corresponding to key 4 (`author`).

Implementations MAY additionally accept a `COSE_Sign1` envelope (RFC 9052) with the same preimage; this exists so that deployments already using COSE — the DMTAP binding among them (§11.2) — need not carry two signature stacks. The bare array form is REQUIRED; COSE support is OPTIONAL.

5.4. Verification

A receiver MUST, in this order, and MUST abort at the first failure:

1. Decode `canonical_bytes` as deterministic CBOR (§4.1). Reject non-canonical encodings with `ERR_NOT_CANONICAL` — do **not** re-encode and continue.
2. Reject key 0 with `ERR_FORBIDDEN_KEY` (§4.5).
3. Check `v` is supported (§4.2).
4. Recompute `id` and compare with key 3. Reject mismatch with `ERR_BAD_ID`.
5. Verify the Ed25519 signature against key 4. Reject with `ERR_BAD_SIG`.
6. Apply the **authorship rule** for the object kind (§5.5). Reject with `ERR_NOT_AUTHORIZED`.

Only after all six does an object enter local state. An implementation MUST NOT display, index, or relay an object that has not passed verification.

Step 1 matters more than it looks. Accepting a non-canonical encoding and re-encoding it before hashing would let two distinct byte strings map to one `id`, which breaks deduplication and lets an attacker produce two objects that a naive implementation treats as the same one.

5.5. Authorship rules

Signature validity is necessary but not sufficient. Each kind constrains *who* may author it:

Kind	Valid author
<code>WorkOrder</code>	Any Principal
<code>Offer</code>	The <code>WorkOrder</code> 's author
<code>Bid</code>	Any Principal the offer's pool admits (§8.4)
<code>Assignment</code>	The <code>WorkOrder</code>'s author only
<code>Progress</code>	The assigned performer, or the issuer
<code>Attestation</code>	Any Principal — but see §9.3 on weighting

An object that is cryptographically valid but violates its authorship rule **MUST** be rejected, not merely down-ranked. In particular, an `Assignment` signed by anyone other than the issuer is not a competing opinion to be resolved by the merge algebra; it is invalid and never enters state. This is what prevents a performer from assigning work to themselves.

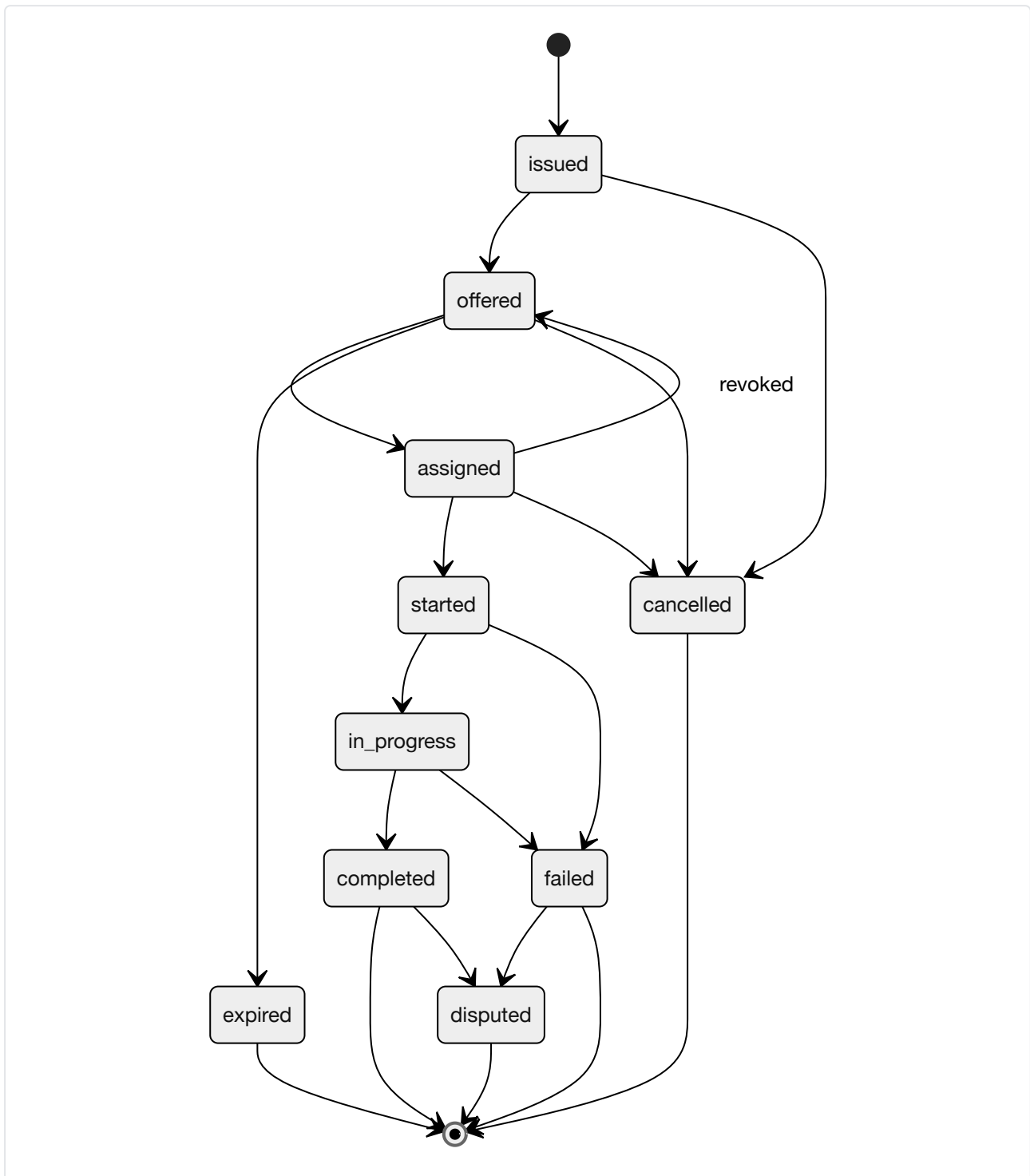
5.6. Referential validation

An object referring to a `WorkOrder` the receiver does not hold **MUST** be retained as **pending**, not rejected. Out-of-order arrival is normal: a `Bid` routinely overtakes the `Offer` that provoked it when they travel by different paths.

A pending object **MUST** be re-validated when its referent arrives, and **MUST** be discarded once the referent's `expires` has passed. Implementations **SHOULD** cap pending storage and evict oldest-first; unbounded pending queues are a denial-of-service surface.

6. Lifecycle

6.1. States



State	Meaning	Set by
issued	Work order exists, not yet offered	Implicit on WorkOrder
offered	Visible to at least one pool	Implicit on Offer
assigned	A performer is committed	Implicit on Assignment
started	Performer has begun (departed, en route, arrived)	Progress

State	Meaning	Set by
<code>in_progress</code>	Substantive work under way	<code>Progress</code>
<code>completed</code>	Work done	<code>Progress</code> + <code>Attestation</code>
<code>failed</code>	Attempted, not achieved	<code>Progress</code>
<code>cancelled</code>	Called off before completion	<code>Progress</code>
<code>expired</code>	<code>expires</code> passed with no assignment	Implicit, computed
<code>disputed</code>	Outcome contested	<code>Attestation</code> with <code>outcome = 3</code>

6.2. Transitions

Transitions are not commands; they are *derived* from the objects present. There is no transition message and no acknowledgement. An implementation computes the current state by folding the object set (§6.3).

`expired` is computed, never signed: any work order whose `expires` has passed without a valid `Assignment` is expired everywhere, simultaneously, with no message required. This is deliberate — expiry is the one state change that must work when the parties cannot reach each other.

`assigned` → `offered` occurs when an `Assignment` carries `revoked = true` (§3.6). The work order returns to its pools and may be re-assigned; earlier bids remain valid unless withdrawn.

6.3. Computing current state

Given the verified object set for one work order, the state is:

1. If any `Progress` reports a **terminal** state (`completed`, `failed`, `cancelled`), the state is the terminal one with the highest `ts`.
2. Otherwise, if a valid non-revoked `Assignment` exists, the state is the highest-`ts` `Progress` state reachable from `assigned`, defaulting to `assigned`.
3. Otherwise, if `now >= expires`, the state is `expired`.
4. Otherwise, if any `Offer` exists, the state is `offered`.
5. Otherwise, `issued`.

The fold is a **pure function of the object set** and therefore order- independent: two replicas holding the same objects compute the same state regardless of arrival order. This is what lets participants sync in any direction, at any time, over any transport, without a session or a handshake.

Implementations MUST discard a `Progress` object whose `state` is not reachable from the state implied by the objects preceding it in `ts` order — a performer cannot report `completed` on a work order they were never assigned. Such an object is not an error to be surfaced; it is simply not applied, and MAY be logged.

6.4. Timeouts

WRAP defines no timers beyond `expires` (§3.3) and `closes` (§3.4). Deployments requiring “assign within N seconds” or “auto-fail if not completed by T” MUST express these as profile fields (§12) and enforce them locally.

This is a deliberate omission. Protocol-level timers require agreement on time and on what a timeout *means*, and both are exactly the kind of coordination WRAP exists to avoid. Local policy over shared clocks.

7. Merge

7.1. The algebra

WRAP state is a set of signed objects. Merging two replicas is **set union**, followed by the read-time fold of §6.3.

Union is commutative, associative and idempotent, so merge is too. Replicas converge regardless of the order, direction, or number of times objects are exchanged, and a duplicate delivery is free rather than harmful. There is no merge *function* to get wrong, because objects are immutable and nothing is ever overwritten.

Where a single current value is needed — the assignment, the current state — it is *derived* by the fold, not stored. Only three merge behaviours exist:

Behaviour	Applies to	Rule
Add-only set	Offer, Bid, Progress, Attestation	Union. Nothing is ever removed.
Single-writer LWW	Assignment	Highest <code>ts</code> among objects by the authorized author.
Immutable	WorkOrder	First-writer-wins by <code>id</code> ; identical <code>id</code> means identical bytes.

Note that last-writer-wins appears exactly once, and only where the protocol has already guaranteed a single legitimate writer (§3.6). WRAP never resolves a conflict between two parties by picking a timestamp winner, because a timestamp winner is arbitrary and someone's work would be discarded.

7.2. Hybrid logical clock

`ts` (key 5) is a hybrid logical clock stamp, encoded as a lexically sortable string:

```
{unix_ms:013d} "-" {counter:04x} "-" {author_hex}
```

- `unix_ms` — 13 zero-padded decimal digits of wall-clock milliseconds.
- `counter` — 4 lowercase hex digits, incremented when minting more than one stamp within the same millisecond.
- `author_hex` — the full 64-character lowercase hex of the author's Ed25519 public key.

Because the encoding is lexically sortable, ordinary string comparison is the total order. No parsing is required to sort.

Minting. On each mint, take `max(now_ms, last_ms)`. If it equals `last_ms`, increment `counter`; otherwise reset `counter` to zero. A node MUST seed `last_ms` from the highest stamp it has already

issued or observed, so a backwards wall-clock step cannot mint a stamp that sorts below existing state.

Observing. On receiving a remote stamp, advance the local clock to `max(local, remote)` before the next mint. This is what makes causally later events sort later even across nodes with skewed clocks.

7.3. Tie-break

Two stamps with equal `unix_ms` and equal `counter` are broken by **byte comparison of `author_hex`**, ascending.

This choice is normative and deliberate. The obvious alternative — breaking ties on a per-node identifier — produces a total order that depends on *which node* wrote an object rather than *who authored* it. Those differ whenever an object is relayed, and two implementations that disagree on the rule converge to different orders while both believing they are correct. Tying on the author public key keeps the order a property of the object itself, so it survives relaying, re-encoding, and storage in a different engine.

An implementation **MUST NOT** introduce a node identifier into the ordering.

7.4. Convergence

Two replicas that have exchanged all objects for a work order **MUST** compute an identical state. Conformance vectors (§15) fix this, including the tie-break case, which is the one most likely to be implemented differently and the least likely to be noticed in testing.

Implementations **SHOULD** expose a **state root** — a content address over the sorted `id` set of all objects for a work order:

```
root = 0x1e || BLAKE3-256( id1 || id2 || ... || idn )    ids sorted ascending
```

Two parties comparing roots can prove they hold byte-identical state rather than merely agreeing on what is currently displayed. This turns “it looks right” into a checkable claim, and it is the cheapest possible sync diagnostic.

7.5. Compaction

An implementation **MAY** discard objects for work orders that reached a terminal state (§6.1) longer ago than a local retention period, provided it retains:

- the `WorkOrder`;
- the final `Assignment`;
- all `Attestation`s.

Attestations **MUST NOT** be discarded on the basis of age. They are the portable record of a worker’s history (§9.4) and are the one thing in WRAP whose value increases with time. A participant who prunes attestations is destroying somebody’s employment record.

7.6. Partition behaviour

While partitioned, a participant continues to issue, bid, assign, progress and attest on the objects it holds. Nothing blocks.

The one operation that cannot be made safe under partition is **assignment of a work order whose issuer is unreachable**, and WRAP resolves this by construction rather than by protocol: only the issuer may assign, so a partitioned performer simply has no assignment to make. When the partition heals, the union includes whatever the issuer decided, and the performer's bids are still there.

The failure mode this avoids is worth naming, because it is the one that makes naive decentralized dispatch unusable: two partitioned dispatchers both assigning the same job to different couriers, both couriers driving to the same restaurant, and no principled way to decide which one was wrong.

8. Pools and discovery

8.1. There is no discovery without infrastructure

State the uncomfortable thing first: **open discovery cannot be done with zero infrastructure**. A plumber's phone cannot learn that a homeowner four suburbs away needs a geyser replaced unless something reachable at a known address told it so. Every system that claims otherwise has merely hidden the infrastructure — BitTorrent has bootstrap nodes and trackers, Nostr has relays, Matrix has homeservers, ActivityPub has instances, IPFS has bootstrap peers.

WRAP therefore does not pursue “no nodes”. It pursues **no privileged nodes**:

- anyone MAY run one, with no permission and no registration;
- they are interchangeable — no special keys, no protocol authority;
- a participant MAY use several simultaneously;
- losing one costs *reachability*, never identity, history, or data.

This is a weaker claim than “fully decentralized” and it is the honest one. It is also the property that actually protects participants, because it is what makes an operator replaceable when they start behaving badly.

8.2. Three paths

Direct (no infrastructure). The issuer knows the performer's key and offers straight to it — a shop and its own drivers, a homeowner and their regular plumber, a landlord and their contractor. This is `Offer.mode = 0` with `pool` set to the issuer's own key. It requires no third party at all, and it covers the large majority of real working relationships.

Pool (the open market). A pool is a Principal that accepts offers and distributes them to a membership. A co-op, a union local, a trade association, a neighbourhood group, a municipality, or a business running a dispatch desk may each operate one. A performer MAY belong to many.

DHT (optional, future). Nothing in this document precludes a distributed hash table for pool discovery. It is not specified in v0 because it is not needed for the deployments that exist, and specifying an unused mechanism is how documents drift from implementations.

8.3. What a pool does and does not do

A pool holds **no authority over the work**. It cannot assign, cannot complete, cannot alter a work order, and cannot forge a bid — every object is signed by its author and a pool has no way to produce a signature it does not hold.

A pool provides exactly two services:

1. **Distribution** — performers learn that a work order exists.
2. **Membership** — the pool decides who receives offers and who may bid (§9).

A pool is therefore a *rendezvous point, not a database of record*. If a pool vanishes tomorrow, every participant retains every work order, assignment, progress event and attestation they hold. They lose the ability to find new counterparties through that pool, and nothing else. Participants **SHOULD** belong to more than one pool for precisely this reason.

8.4. Membership

Pool membership is expressed as an `Offer` reaching a performer and a `Bid` being accepted from one. WRAP does not define a membership object, an application flow, or an eviction mechanism in v0.

This is a deliberate scope boundary, not an oversight. How a pool decides who belongs — open registration, vouching, licensing checks, union membership, paying dues, geographic residency — is *governance*, and governance is the part of a labour platform that most deserves to be decided by the people affected rather than by a protocol author. WRAP's job is to ensure that whatever a pool decides, it cannot use that position to seize a worker's identity or history.

8.5. Privacy

An `Offer` carries a work order to a pool, and work orders contain customer addresses and commercial terms. A pool operator therefore sees the work it distributes. Implementations **MUST** make this visible to users and **MUST NOT** describe pool distribution as private.

Two mitigations are available and both are **RECOMMENDED** where the deployment allows:

- **Coarse offers.** Publish only what is needed to bid — an area, a window, a compensation range — and disclose exact addresses in the `Assignment`, which goes only to the assigned performer.
- **Direct offers.** Where the relationship already exists, `mode = 0` bypasses pools entirely and no third party sees anything.

Sealed-bid mode (`mode = 2`) additionally hides bids from other bidders until `closes`; it does not hide them from the pool.

8.6. Why not gossip

Broadcasting every work order to every participant would publish customer addresses and commercial volumes to strangers, create an unbounded spam surface, and scale poorly for a workload whose natural scope is a few kilometres and a few hours. A mesh also removes the one place where membership policy can live, which does not eliminate gatekeeping — it merely relocates it to whoever writes the client's filtering rules.

Point-to-point plus pools keeps the sensitive data narrow, keeps the trust decision explicit, and keeps the failure domain small.

9. Trust

9.1. WRAP is not trustless

Anyone can generate an Ed25519 keypair in microseconds. An open pool with no gatekeeper will therefore accumulate fake performers as fast as someone cares to script them, and a reputation system built on unweighted attestations can be inflated by a Sybil signing praise for itself.

No protocol solves this. Proof-of-work prices out the poor before it prices out the determined; stake-based schemes require capital and exclude exactly the workers this protocol exists to serve; and “web of trust” reduces, in practice, to somebody vouching. WRAP does not claim a solution. It makes the trust source **explicit and replaceable** instead of pretending it is absent.

9.2. Curated pools

The RECOMMENDED deployment is a **curated pool**: membership is vouched for by the pool operator, who is accountable to its members.

This is federated trust, not trustlessness, and it maps onto structures that already exist and already do this work — a courier co-operative, a union local, a trade association that verifies plumbing licences, a municipality maintaining a register, a business dispatching its own staff. These bodies already perform identity verification for their own reasons; WRAP lets them express it without also owning the workers’ data.

The protocol keeps the operator honest in one specific way: because a pool holds no authority over work, no ability to forge signatures, and no exclusive copy of any record (§8.3), a pool that abuses its position can be left. The worker walks out with every attestation intact. That is the entire safeguard, and it is a real one — it converts “trust the platform” into “choose a gatekeeper you can fire”.

9.3. Weighting attestations

An **Attestation** (§3.8) is meaningful only in the light of who signed it. Implementations **MUST NOT** compute a reputation as an unweighted count.

At minimum:

- A **self-attestation** — where `author == subject` — **MUST** be excluded.
- Attestations from Principals with no independent history **SHOULD** be weighted near zero, since they are the cheapest thing an attacker can manufacture.
- Attestations **SHOULD** be weighted by the attester’s own standing in the pool evaluating them, and pools **SHOULD** publish how they weight.
- An attestation without a corresponding **Assignment** naming both parties on the same work order **MUST** be ignored. You cannot review work you were never party to.

9.4. Reputation is inverted

The naive design — each performer publishes their own reputation feed — is broken, and it is worth being explicit about why, because it is the design most implementers reach for first. A performer who controls their feed will omit the jobs that went badly. A feed that can be silently truncated carries no information.

WRAP inverts it:

Issuers publish outcomes about performers. Performers publish outcomes about issuers. Neither publishes about itself.

A performer's reputation is therefore an *aggregate over attestations signed by counterparties*, which the performer cannot suppress because they do not hold the signing keys and do not control the copies. The performer carries pointers to their attestations for convenience; the attestations' authority comes from their authors.

Symmetry matters here and is not decoration. Issuers can be bad counterparties — cancelling after a courier has driven across town, disputing completed work, setting impossible windows. A protocol that lets only one side rate the other reproduces the power asymmetry it was meant to remove.

9.5. Append-only publication

Where attestations are published as a feed, that feed **SHOULD** be append-only and hash-chained: each entry carries a sequence number and the hash of its predecessor, so an omission or reordering is detectable by any reader who has seen an earlier head.

Readers **SHOULD** retain the highest sequence number seen per publisher and reject a later head with a lower sequence, which detects rollback. Without this, a publisher can serve different histories to different readers.

This is the one place WRAP recommends a specific external mechanism: the structure is that of an RFC 6962 style log, and the DMTAP-PUB feed object (§11.2) is a conformant instance of it that requires no mesh and runs over plain HTTPS. Implementations **MAY** use any equivalent construction.

9.6. Disputes

WRAP records disputes; it does not resolve them. An `Attestation` with `outcome = 3` marks a contested outcome, and both parties' attestations remain in the record permanently, each signed by its author.

Resolution is a social and legal matter for the pool, the parties, or a court. A protocol that adjudicated disputes would be a protocol that could be captured by whoever controls the adjudication, which is the thing being removed.

10. Fulfilment

10.1. The problem

“Delivered”, signed only by the performer who was paid to deliver it, is worth nothing in a dispute. The issuer was not present at the handoff and the beneficiary — the customer receiving the parcel, the tenant whose tap was fixed — usually holds no key and never will.

WRAP needs a proof that a party who was *not there* can check afterwards, without requiring the beneficiary to run software.

10.2. Handoff codes

The RECOMMENDED mechanism is a **handoff code**: a short secret known to the issuer and the beneficiary, presented at the moment of handoff and included in the completing attestation.

1. On assignment, the issuer generates a code — 4 to 6 digits is sufficient — and delivers it to the beneficiary out of band (printed on the receipt, sent by message, read aloud on the phone).
2. The issuer includes `commit = BLAKE3-256(code || order_id)` in the `Assignment` (§3.6, `terms`), so the commitment is published *before* the handoff and cannot be constructed afterwards to match whatever was typed.
3. At handoff, the beneficiary tells the performer the code.
4. The performer includes the code in the `proof` map of their completing `Attestation` (§3.8).
5. Any party recomputes the hash and checks it against the commitment.

The performer cannot fabricate a completion, because they never learn the code except from the beneficiary. The issuer cannot deny a completion that happened, because they published the commitment beforehand and the code verifies against it.

This is deliberately the mechanism couriers already use. It requires no app, no key, and no literacy beyond reading four digits, and it degrades to a phone call when everything else fails.

10.3. Proof map

Key	Name	Type	Notes
1	<code>kind</code>	uint	<code>0</code> code, <code>1</code> signature, <code>2</code> photo, <code>3</code> geo, <code>4</code> none
2	<code>code</code>	tstr	The handoff code, for <code>kind = 0</code>
3	<code>sig</code>	bstr	Beneficiary signature, for <code>kind = 1</code>
4	<code>ref</code>	tstr	Content address or URL of a photo, for <code>kind = 2</code>
5	<code>at</code>	map	<code>Place</code> (§3.9), for <code>kind = 3</code>
6	<code>note</code>	tstr	Why proof is absent, for <code>kind = 4</code>

10.4. Weaker proofs

Beneficiary signature (`kind = 1`) is the strongest available and the least practical: it requires the beneficiary to hold a key. Use it where the beneficiary is itself a business — a restaurant receiving a supplier delivery, a landlord accepting completed work.

Photo (kind = 2) proves something was photographed, not that it was delivered to the right person. It is evidence, not proof, and implementations **MUST NOT** present it as verification.

Geolocation (kind = 3) proves a device reported coordinates. Device location is trivially falsified on both major mobile platforms. Implementations **MUST NOT** treat geolocation as proof of anything, and **SHOULD NOT** present it to users as verification. It is useful for operations — routing, ETAs, dispute context — and useless for adjudication.

None (kind = 4) is legitimate and **MUST** be supported. Much real work has no verifiable handoff: a bin is emptied, a garden is mowed, nobody is home. The honest encoding of “we have no proof” is **kind = 4** with a note, not a weaker proof dressed up as a strong one.

10.5. Partial and multi-visit work

Trades work frequently completes across several visits, and a first visit may be a quotation rather than work at all. Implementations **MUST NOT** assume one assignment means one visit.

Each visit is a **Progress** object (§3.7). Only the final one carries the terminal state, and only then is a completing **Attestation** appropriate. A work order requiring a return visit stays **in_progress** between them; this is the normal case for trades and needs no special handling.

Where a first visit produces a revised price, the performer records it as a **Progress** object carrying the new terms in **body**, and the issuer accepts by issuing a fresh **Assignment** with updated **terms**. Neither party can silently change agreed compensation, because both objects are signed and both remain in the record.

11. Transport bindings

11.1. Binding A — HTTP (REQUIRED)

Every conformant implementation **MUST** support the HTTP binding. It is the baseline that makes WRAP implementable without adopting anything else.

11.1.1. Endpoints

Method	Path	Purpose
GET	<code>/.well-known/wrap/identity</code>	Principal's public key and supported versions
GET	<code>/wrap/v0/vector</code>	Version vector — what this node holds
POST	<code>/wrap/v0/objects</code>	Push signed objects
POST	<code>/wrap/v0/pull</code>	Request objects the caller lacks
GET	<code>/wrap/v0/order/{id}</code>	All objects for one work order

Requests and responses carry `application/wrap+cbor`: a definite-length CBOR array of signed-object envelopes (§5.3).

A sync round is **stateless and symmetric** — push what the peer lacks, then pull what we lack. Nodes hold no per-peer session state, so any node may relay any other node's objects and a round may be abandoned and retried at any point without reconciliation. Implementations **SHOULD** cap a batch at 2000 objects.

11.1.2. Request authentication

The transport is authenticated independently of object signatures, because a transport signature answers a different question: not “who wrote this object” but “who is talking to me right now”.

The caller signs a canonical envelope:

```
sigbase = "WRAP-v0/http" || 0x00 || method || 0x00 || path || 0x00 ||  
          BLAKE3-256(body) || 0x00 || timestamp || 0x00 || nonce
```

sent as headers `Wrap-Key`, `Wrap-Ts`, `Wrap-Nonce`, `Wrap-Sig`.

A receiver **MUST** reject a request whose `timestamp` is more than **300 seconds** from local time, and **MUST** maintain a nonce cache with a TTL of at least twice that window, rejecting repeats. Both checks are **REQUIRED**: freshness alone permits replay inside the window.

Note that transport authentication is not required for correctness of the *data* — every object is independently signed and verified (§5), so a compromised transport cannot forge state. It exists to control who may consume resources and to make relaying accountable.

11.1.3. Pairing

A node that has never seen a peer has no key to verify against. WRAP uses trust-on-first-use bootstrapped by a **pairing secret**: an unenrolled caller proves knowledge of a shared secret, which authorizes recording the public key it presents. Thereafter the key alone authenticates and the secret is irrelevant to that peer.

Implementations **MUST** default to rejecting unenrolled peers, and **MUST** require explicit operator action to accept a pairing. Revocation is deletion of the peer record; full revocation is deletion plus rotation of the pairing secret.

11.1.4. Offline transports

Because objects are self-authenticating (§5.1), any medium that moves bytes is a valid transport. Implementations **SHOULD** support **file exchange**: each node appends only its *own* objects to a file named for its Principal, and imports every other file it finds.

Since no two nodes ever write the same file, a shared folder — a synced drive, a NAS, or a USB stick carried between sites — becomes a transport with no possibility of a write conflict. This is not a curiosity: it is the transport that works during a power cut, and for many deployments it is the only one that works reliably.

11.2. Binding B — DMTAP substrate (OPTIONAL)

WRAP objects **MAY** be carried as operations in the DMTAP substrate sync algebra, and attestation feeds (§9.5) **MAY** be published as DMTAP-PUB feed objects over plain HTTPS at `/.well-known/dm-tap-pub/*`.

This binding is **OPTIONAL** and nothing in the core depends on it. Two properties make it nearly free:

- WRAP identities are Ed25519 public keys, and the DMTAP identity key is also Ed25519, so a DMTAP participant is already a valid WRAP Principal (§2.1).

- The tie-break rule of §7.3 is byte comparison of the author public key, which is the substrate's rule. An implementation that follows §7.3 can move state between the two without reordering it.

Had WRAP tied on a node identifier instead, this binding would have been lossy — the two engines would converge to different orders while each believed it was correct. This is why §7.3 is normative rather than advisory.

Deployments **MUST** choose one merge engine per deployment and **MUST NOT** mix them. Two engines with different total orders cannot share a replica set, and a gradual migration between them is specifically the thing that does not work.

11.3. Other bindings

Nothing prevents a further binding — a message queue, a mesh, a Bluetooth link between a courier's phone and a shop tablet. A binding is conformant if it delivers intact signed objects and preserves the rules of §5.4. Bindings do not need to preserve order, deliver exactly once, or guarantee delivery; the merge algebra (§7) tolerates reordering, duplication and loss by construction.

12. Profiles

12.1. What a profile is

The core (§2–§11) is domain-neutral. A **profile** specialises it for a kind of work by defining:

- a profile identifier, used in `WorkOrder.profile` (§3.3);
- the meaning of `Place.role` values;
- profile-specific fields in `body` maps, using CBOR keys 32 and above (§4.5);
- OPTIONAL additional object kinds in the range `0x40 – 0x7f`;
- expected fulfilment proof (§10).

A profile **MUST NOT** redefine core semantics. It cannot change who may assign, alter the merge rules, or introduce a state outside §6.1. An implementation that does not recognise a profile **MUST** still be able to verify, store, relay and merge its objects — it simply cannot render them meaningfully. This is what keeps profiles additive rather than forking.

12.2. `delivery/v0`

Moving an item between two places, immediately.

Field	Key	Notes
<code>vehicle</code>	32	<code>"foot"</code> , <code>"bicycle"</code> , <code>"motorcycle"</code> , <code>"car"</code> , <code>"van"</code>
<code>items</code>	33	Array of <code>{label, qty, note}</code>
<code>value</code>	34	Declared value, minor units, for insurance and handling
<code>constraints</code>	35	<code>"chilled"</code> , <code>"fragile"</code> , <code>"upright"</code> , <code>"age-check"</code>

- `Place.role` — `"pickup"`, `"dropoff"`.
- `Window.kind` — `0` (immediate) is typical.
- Fulfilment — handoff code (§10.2) RECOMMENDED.

- `Progress` states used: `started` (departed), `in_progress` (collected), `completed` (handed over).
- Live location is conveyed as `Progress` objects carrying `at`. These are append-only and high-volume; implementations SHOULD rate-limit them and SHOULD compact them aggressively after completion (§7.5).

Vocabulary here is deliberately aligned with prior art in the community-owned delivery space (§16) so that a bridge to those systems is a mapping rather than a translation.

12.3. `trades/v0`

Skilled work at a site, scheduled, often multi-visit, frequently re-quoted after inspection.

Field	Key	Notes
<code>trade</code>	32	"plumbing", "electrical", "hvac", "carpentry", ...
<code>licence</code>	33	Required credential, e.g. "za:pirb"
<code>visit</code>	34	<code>0</code> quotation, <code>1</code> work, <code>2</code> follow-up
<code>materials</code>	35	Array of {label, qty, cost}
<code>access</code>	36	Site access constraints, key collection, occupant presence

- `Place.role` — "site".
- `Window.kind` — `1` (scheduled appointment) is typical, and this is the field that makes trades expressible at all.
- Fulfilment — beneficiary signature (§10.4) where the client is present; `kind = 4` with a note where they are not.
- Multi-visit and re-quotation follow §10.5.

Trades is included as a second profile *in v0 specifically* because designing against delivery alone produces a core that silently assumes work is immediate, single-visit and fixed-price. Every one of those assumptions is false for a plumber, and each would have been baked into the state machine had the second profile been deferred.

12.4. Other applicable domains

Non-normative. These are recorded to show which parts of the core are load-bearing, and where a future profile would need to extend it.

Domain	What it exercises
Field service — ATMs, vending, telecoms towers, lifts	Scheduled windows, licences, parts inventory, assets that are themselves the "place"
Home & community care — nursing visits, community health workers	Recurring visits, strict privacy on <code>detail</code> , <code>kind = 4</code> proof (a visit often has no artefact)
Mutual aid & disaster response	Zero-compensation work orders, surge volume, pools that form and dissolve within days
Municipal & citizen reporting — potholes, streetlights, waste	The <i>citizen</i> is the issuer; a municipality is the performer. Inverts the usual power direction and is a good test that the model is not merchant-centric

Domain	What it exercises
Agricultural contracting — harvest crews, tractor hire	Seasonal pools, crew-sized assignment (one assignment, many humans), weather-driven cancellation
Roadside assistance & towing	Immediate dispatch with geographic urgency; overlaps <code>delivery/v0</code> closely
Medical courier — lab samples, pharmacy	Chain-of-custody: every handoff needs proof, not only the last one
Inspection & assessment — building, insurance, safety	The deliverable is a <i>document</i> , so fulfilment is a content reference rather than a handoff
Remote/freelance work — translation, design, code	No <code>Place</code> at all. A useful test that geography is optional in the core, which it is: <code>places</code> is a MAY field
Event & film crew calls	Many performers per work order, call times, role-specific requirements

Two observations from this list shaped the core.

Geography is optional. `places` is a MAY field precisely because remote work is a legitimate domain. An implementation that requires coordinates has narrowed the protocol for no reason.

The issuer is not necessarily a business. A resident reporting a pothole is an issuer; a municipality is the performer. Nothing in WRAP privileges commercial issuers, and any profile that assumes one has made an error.

12.5. Registering a profile

Profiles are identified by `"name/vN"` in `WorkOrder.profile`. v0 defines no registry. Deployments SHOULD use a namespaced identifier for anything not defined here — `"example.org:towing/v0"` — so that independent profiles do not collide before a registry exists.

13. Errors

13.1. Codes

Code	Name	Meaning
<code>0x0101</code>	<code>ERR_FORBIDDEN_KEY</code>	Object contains key 0 (§4.5)
<code>0x0102</code>	<code>ERR_NOT_CANONICAL</code>	Encoding is not deterministic CBOR (§4.1)
<code>0x0103</code>	<code>ERR_BAD_ID</code>	Recomputed <code>id</code> does not match key 3 (§4.3)
<code>0x0104</code>	<code>ERR_BAD_SIG</code>	Ed25519 verification failed (§5.3)
<code>0x0105</code>	<code>ERR_T00_LARGE</code>	Object exceeds 65 536 bytes (§4.6)
<code>0x0106</code>	<code>ERR_UNSUPPORTED_VERSION</code>	<code>v</code> not supported (§4.2)
<code>0x0201</code>	<code>ERR_NOT_AUTHORIZED</code>	Author violates the kind's authorship rule (§5.5)
<code>0x0202</code>	<code>ERR_NOT_ISSUER</code>	<code>Assignment</code> not authored by the work order's issuer (§3.6)
<code>0x0203</code>	<code>ERR_EXPIRED</code>	Work order's <code>expires</code> has passed (§3.3)
<code>0x0204</code>	<code>ERR_CLOSED</code>	Offer's <code>closes</code> has passed (§3.4)

Code	Name	Meaning
0x0301	ERR_UNKNOWN_REFERENT	Referenced object not held — object is pending, not rejected (§5.6)
0x0302	ERR_UNREACHABLE_STATE	<code>Progress</code> state not reachable from current state (§6.3)
0x0401	ERR_STALE_REQUEST	Transport timestamp outside the freshness window (§11.1.2)
0x0402	ERR_REPLAY	Transport nonce already seen (§11.1.2)
0x0403	ERR_UNPAIRED	Caller's key is not enrolled (§11.1.3)

Codes `0x0900` and above are reserved for profile-specific errors.

13.2. Reporting

Errors are diagnostic, not protocol state. An implementation **MUST NOT** create a WRAP object to report an error, and **MUST NOT** let a rejected object affect merge outcomes.

Over the HTTP binding, a rejection is reported as `400` with a CBOR body `{1: code, 2: message}`. A batch containing some invalid objects **MUST** accept the valid ones and report the rejected subset by `id`; rejecting an entire batch because one object failed lets a single malformed object block a peer's whole queue.

13.3. Silence

Three conditions are explicitly **not** errors and **MUST NOT** be reported:

- an **unknown kind** (§4.4) — ignored silently;
- an **unknown field** (§4.4) — preserved and ignored;
- an **unknown profile** (§12.1) — stored, relayed, merged, not rendered.

Each of these is a forward-compatibility path. An implementation that errors on them converts every future extension into a breaking change, which is the most common way an extensible format stops being extensible.

14. Security considerations

14.1. What WRAP protects

- **Authorship.** Every object is signed; nobody can put words in another party's mouth. In particular no performer can assign work to themselves (§5.5) and no pool can forge a bid (§8.3).
- **Integrity.** Objects are content-addressed and immutable; tampering changes the `id` and fails verification.
- **Non-repudiation.** A signed attestation cannot be disowned later, by either party.
- **Ownership of history.** Attestations are held by their authors and their subjects, not by an operator, so leaving a pool costs nothing but reach (§9.2).

14.2. What WRAP does not protect

Stated plainly, because a security section that lists only strengths is marketing.

- **Confidentiality against a pool.** A pool operator sees the work orders it distributes, including addresses and terms (§8.5). WRAP does not encrypt offers in v0.
- **Metadata privacy.** Who works for whom, how often, and where is visible to counterparties and to any pool in the path. WRAP is not an anonymity system and MUST NOT be described as one.
- **Location integrity.** Device geolocation is trivially falsified. It is operational data, never proof (§10.4).
- **Key loss.** There is no rotation or recovery in v0 (§2.4). A lost key is a lost identity and a lost work history.
- **Sybil resistance.** Not solved; made explicit and delegated to pool curation (§9.1).
- **Availability.** A pool can refuse to distribute a participant's work. WRAP guarantees only that this costs them nothing but that pool.

14.3. Misbehaving parties

A **malicious issuer** can sign two `Assignment` objects for one work order with the same `ts`. The merge algebra will pick one deterministically, but two performers may already have started. This is not a distributed-systems failure — it is a party misbehaving in a way that is signed, permanent and attributable. Both performers hold the signed evidence and can attest accordingly (§9.4). WRAP's answer to bad behaviour is durable evidence, not prevention.

A **malicious performer** can report `completed` without doing the work. The handoff commitment (§10.2) defeats this whenever a beneficiary is present, and `kind = 4` honestly marks the cases where it cannot.

A **malicious pool** can withhold offers, favour members, or leak commercial data. It cannot forge, assign, complete, or destroy history. The mitigation is exit, and it is only real if participants belong to more than one pool — implementations SHOULD encourage this in the interface, not merely permit it.

A **malicious peer** can flood a node with well-formed objects. Implementations MUST bound pending storage (§5.6), MUST enforce the size limit (§4.6), and SHOULD rate-limit per authenticated peer key.

14.4. Replay

Object replay is harmless by construction: objects are immutable and merge is idempotent, so a re-delivered object changes nothing.

Transport replay is prevented by the timestamp-and-nonce rule of §11.1.2. Both halves are REQUIRED; a freshness window alone permits replay inside it, which is the mistake this rule exists to prevent.

14.5. Denial of service

The `expires` field (§3.3) is REQUIRED for a security reason as well as a usability one: without mandatory expiry, an attacker can permanently inflate every pool with work orders that never resolve, and no participant can distinguish a stale pool from a busy one.

Implementations MUST discard expired work orders and their pending objects, and MUST NOT allow retention policy to be set to unbounded.

14.6. Privacy of beneficiaries

The most sensitive data in a WRAP deployment does not belong to any participant: it is the customer's home address, and the customer never consented to a protocol.

Implementations SHOULD publish coarse offers and disclose exact addresses only in the `Assignment`, which reaches one performer (§8.5). Implementations SHOULD compact location trails after completion (§7.5). Implementations MUST NOT publish beneficiary addresses in attestation feeds, which are long-lived, widely replicated, and designed never to be deleted.

15. Conformance

15.1. Vectors are normative

The conformance vectors in `conformance/wrap_vectors.json` are **normative and take precedence over the prose of this document** wherever the two disagree.

This inversion is deliberate. Prose cannot tell an implementer that they broke a tie the wrong way, sorted a map incorrectly, or omitted a domain-separation tag — all three produce code that passes every test the author thought to write, interoperates with itself perfectly, and fails silently against a second implementation. Vectors catch exactly this class of error, and they are the difference between protocols that acquire independent implementations and protocols that acquire admirers.

A claim of WRAP v0 conformance means: all vectors pass, with no skips. An implementation that skips vectors MUST say which and why.

15.2. Required coverage

Group	What it fixes
<code>encode</code>	Deterministic CBOR for every object kind; key ordering; shortest-form integers
<code>id</code>	<code>id</code> computation over canonical bytes, including omission of key 3
<code>sign</code>	Preimage construction with the domain tag; signature verify pass and fail
<code>reject</code>	Key 0; non-canonical encoding; oversize; bad id; bad signature; unsupported version
<code>authorship</code>	<code>Assignment</code> by a non-issuer rejected; <code>Bid</code> by any principal accepted
<code>hlc</code>	Mint monotonicity; observe advancement; backwards-clock seeding
<code>tiebreak</code>	Equal <code>unix_ms</code> and <code>counter</code> , resolved by author public key
<code>merge</code>	Union convergence; idempotent redelivery; reordered delivery; three-way relay
<code>fold</code>	State computation from an object set, including unreachable-state discard
<code>expiry</code>	Computed expiry with no message exchanged
<code>forward</code>	Unknown kind ignored; unknown field preserved through re-encode; unknown profile stored
<code>proof</code>	Handoff commitment verifies; wrong code fails

15.3. The cases most likely to be got wrong

Implementers SHOULD run these first.

1. **Tie-break on author key (§7.3).** The most likely divergence in the whole document, and invisible until a second implementation exists.
2. **Preserving unknown fields through a re-encode (§4.4).** Dropping them invalidates the signature for everyone downstream, and the implementation that dropped them will not be the one that reports the failure.
3. **Rejecting rather than repairing non-canonical encodings (§5.4 step 1).** Repairing lets two byte strings share one `id`.
4. **Pending rather than rejecting unknown referents (§5.6).** Rejecting makes out-of-order delivery lossy, and out-of-order delivery is normal.
5. **Computed expiry (§6.2).** Implementations that wait for an expiry message will hold work orders forever when a peer is unreachable.

15.4. Interoperability

Two implementations are interoperable at v0 if, given the same object set delivered in any order, they compute the same state (§6.3) and the same state root (§7.4).

State root equality is the RECOMMENDED interoperability test because it fails loudly on divergence that a state comparison would hide — two replicas can display identical current state while holding different histories, and the difference surfaces only later, usually during a dispute.

16. References

16.1. Normative

Reference	Used for
RFC 2119 / RFC 8174 (BCP 14)	Requirements language
RFC 8032	Ed25519 signatures (§5)
RFC 8949	CBOR, including deterministic encoding §4.2.1 (§4)
BLAKE3	Object identifiers, commitments, state roots (§4.3, §7.4, §10.2)
RFC 9052	COSE_Sign1, OPTIONAL alternative envelope (§5.3)
RFC 6962	Hash-chained log structure for attestation feeds (§9.5)

16.2. Informative

OpenCourier — *An Open Protocol for Building a Decentralized Ecosystem of Community-owned Delivery Platforms*, CHI 2026 Extended Abstracts. arxiv.org/abs/2511.02455

The closest prior art, and the source of the delivery vocabulary in §12.2. OpenCourier proposes a three-layer architecture — registry, instance-app, instance-requester — for worker-owned delivery platforms, and its framing of worker ownership as the goal rather than decentralization for its own sake directly informed §9.2 and §8.1.

It differs from WRAP in two ways that matter. It is a position paper with embedded protocol sketches rather than a specification: there is no formal wire format, no public reference repository, and no defined trust or reputation-portability mechanism. And its architecture is always-online federated REST

instances, which cannot make progress during a partition — the constraint that drives most of WRAP's design (§1.4). The two are complementary rather than competing; a bridge would be a mapping, not a translation.

OpenTripModel — lightweight logistics trip vocabulary. opentripmodel.org Informative for **Place** and **Window** shaping (§3.9, §3.10).

GS1 EPCIS — supply-chain event capture (what / when / where / why). Informative for the append-only event model of **Progress** (§3.7). EPCIS is far heavier than WRAP needs and assumes centralized capture.

OASIS UBL — Universal Business Language, including Despatch Advice and Waybill. Informative for **Compensation** (§3.11). XML and enterprise-oriented; adopted for vocabulary only.

W3C ActivityStreams 2.0 — the **Offer** / **Accept** / **Reject** verb vocabulary parallels §3 closely. Not adopted: ActivityStreams has no merge semantics and no offline story, both of which are the substance of WRAP.

DMTAP — Decentralized Message Transfer & Access Protocol, and its substrate capabilities (Identity, Feeds & Blobs, Sync, Roles, Wake). [draft-dmtap-01](#)

Source of the 8-word key name convention (§2.3), the CRDT sync algebra underlying binding B (§11.2), and the append-only feed structure recommended for attestations (§9.5). WRAP depends on none of it: DMTAP is a binding WRAP happens to fit, not a substrate WRAP requires. The **0x1e** multihash prefix, domain-separated hashing, and the reserved-forbidden-key technique of §4.5 are adopted from its public-objects profile.

FlowStock — inventory and sales across branches, offline-first. github.com/vul-os/flowstock

The HLC stamp format (§7.2), stateless symmetric sync rounds, TOFU pairing (§11.1.3), and the file-exchange transport (§11.1.4) are taken from its working implementation rather than designed here. Its treatment of quantities as an append-only movement log summed at read time — rather than a stored counter — is the pattern that makes concurrent offline work converge correctly, and it is the direct ancestor of §7.1.

16.3. Acknowledgements

The problem framing owes a debt to the platform cooperativism literature, particularly the argument that the defect of gig platforms is ownership rather than technology. WRAP takes that seriously enough to try to make the technology stop being an obstacle to it.